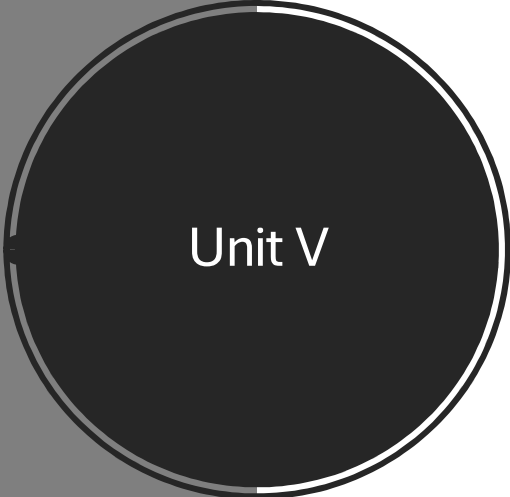


A dark gray circle with a thin white border, containing the text 'Unit V' in white. The circle is positioned on the left side of the slide, overlapping a dark gray rectangular background.

Unit V

Steps to Build a Machine Learning Model

Model Training and Testing Process

Improving Model Accuracy and Generalization

Random Forest Classifier

Implementing K- Means and DBSCAN from Scratch

Performance Metrics – Accuracy, Precision, Recall

Sensitivity and Specificity

ROC Curve and AUC - Visualization

Bias- Variance Tradeoff

Model Comparison and Selection for Deployment

Steps to Build a Machine Learning Model

Machine Learning (ML) helps computers learn patterns from data and make intelligent predictions.
Building an ML model involves several systematic steps to ensure accuracy, efficiency, and reliability.

Key Steps:

Data Collection:

Gather relevant and high-quality data from sources like databases, APIs, or web scraping.

Good data quality improves the model's accuracy and generalization.

Data Preprocessing & Cleaning:

Transform raw data into usable form by removing missing or noisy values, encoding categorical variables, and scaling features.

Clean data ensures better training and performance.

Model Selection:

Choose the right algorithm based on the problem type:

- Regression → Predict continuous values
- Classification → Predict categories
- Clustering → Group similar data points

Model Training:

Feed preprocessed data into the chosen algorithm.

The model learns patterns by adjusting internal parameters using techniques like gradient descent.

Model Evaluation:

Assess performance using metrics such as:

- Regression → MAE, MSE, RMSE, R^2
- Classification → Accuracy, Precision, Recall, F1-score, AUC-ROC

Tuning & Optimization:

Improve model accuracy by adjusting hyperparameters using Grid Search, Random Search, or Cross-Validation.

Model Deployment:

Integrate the optimized model into a real-world application for live predictions.

Use tools like **Docker** and **Kubernetes** for efficient scaling and updates.

Conclusion:

ML model building is an iterative process—from collecting and cleaning data to deploying a tuned model that provides real-time, accurate predictions.

Training Set, Testing Set, Validation Set

Purpose of Data Splitting:

To evaluate how well a trained model generalizes to unseen data using the `train_test_split()` function from Scikit-learn.

Training Set:

- Used to **train** the model — it learns patterns and relationships from this data.
- Usually **60–70%** of total data.
- Quality and diversity of data directly affect model performance.

Example:

```
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8)
```

→ 80% training, 20% testing.

Testing Set:

- Used **after training** to evaluate the model's performance.
- Usually **20–25%** of data.
- Independent from training data.
- Helps detect **overfitting** if accuracy on training > testing.

Example:

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

Set Type	Purpose	Data %	Used For
Training	Learn model parameters	60–70%	Model Training
Validation	Tune model	10–15%	Hyperparameter tuning
Testing	Evaluate model	20–25%	Final performance check

Validation Set:

- Used to **fine-tune hyperparameters** and assess model generalization.
- The model **does not learn** from this data — it's for evaluation only.
- Usually **10–15%** of total data.

Purpose:

- Helps avoid **overfitting** and improves **model tuning**.
- If validation accuracy > training accuracy → good generalization.

Splitting Example:

```
x_train, x_temp, y_train, y_temp = train_test_split(x, y, train_size=0.8)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, test_size=0.5)
```

→ 80% Training | 10% Validation | 10% Testing

Implement Random Forest

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import
    RandomForestClassifier
from sklearn.metrics import accuracy_score,
    classification_report
import warnings
warnings.filterwarnings('ignore')
titanic_data = pd.read_csv('titanic.csv')
titanic_data =
    titanic_data.dropna(subset=['Survived'])
X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch',
    'Fare']]
y = titanic_data['Survived']
X.loc[:, 'Sex'] = X['Sex'].map({'female': 0, 'male': 1})
X.loc[:, 'Age'].fillna(X['Age'].median(), inplace=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

```
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)
y_pred = rf_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y
    _pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n", classification_r
    ep)
sample = X_test.iloc[0:1]
prediction = rf_classifier.predict(sample)
sample_dict = sample.iloc[0].to_dict()
print(f"\nSample Passenger: {sample_dict}")
print(f"Predicted Survival: {'Survived' if
    prediction[0] == 1 else 'Did Not Survive'}")
```

Implement Random Forest For Regression

```
import pandas as pd

from sklearn.datasets import
    fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.ensemble import
    RandomForestRegressor

from sklearn.metrics import mean_squared_error,
    r2_score

california_housing = fetch_california_housing()

california_data =
    pd.DataFrame(california_housing.data,
        columns=california_housing.feature_names)

california_data['MEDV'] = california_housing.target

X = california_data.drop('MEDV', axis=1)

y = california_data['MEDV']
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
    y, test_size=0.2, random_state=42)

rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)

rf_regressor.fit(X_train, y_train)

y_pred = rf_regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

single_data = X_test.iloc[0].values.reshape(1, -1)

predicted_value = rf_regressor.predict(single_data)

print(f"Predicted Value: {predicted_value[0]:.2f}")

print(f"Actual Value: {y_test.iloc[0]:.2f}")

print(f"Mean Squared Error: {mse:.2f}")

print(f"R-squared Score: {r2:.2f}")
```

Implement DBSCAN

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA
X = pd.read_csv('..input_path/CC_GENERAL.csv')
# Dropping the CUST_ID column from the data
X = X.drop('CUST_ID', axis = 1)
# Handling the missing values
X.fillna(method='ffill', inplace = True)
print(X.head())

# Scaling the data to bring all the attributes to a comparable level
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Normalizing the data so that
# the data approximately follows a Gaussian distribution
X_normalized = normalize(X_scaled)
# Converting the numpy array into a pandas DataFrame
X_normalized = pd.DataFrame(X_normalized)
```

```
pca = PCA(n_components = 2)
X_principal = pca.fit_transform(X_normalized)
X_principal = pd.DataFrame(X_principal)
X_principal.columns = ['P1', 'P2']
print(X_principal.head())
```

```
# Numpy array of all the cluster labels assigned to each data
point
db_default = DBSCAN(eps = 0.0375, min_samples =
3).fit(X_principal)
labels = db_default.labels_
```

```
db = DBSCAN(eps = 0.0375, min_samples =
50).fit(X_principal)
labels1 = db.labels_
```

DBSCAN vs K-Means Clustering

- **When to Use DBSCAN Over K-Means:**

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) and K-Means are both clustering algorithms but differ in how they identify clusters.

Use DBSCAN when the data has **irregular (non-spherical) shapes** or the **number of clusters is unknown**.

- **Summary:**

DBSCAN is ideal for data with complex, uneven cluster structures and noise.

K-Means works best for clean data with well-separated, spherical clusters.

Aspect	DBSCAN	K-Means
Number of Clusters	Not required to be specified	Must be specified beforehand
Cluster Shape	Can form arbitrary-shaped clusters	Assumes spherical or convex clusters
Noise & Outliers	Handles noise and outliers effectively	Sensitive to outliers — can distort clusters
Parameters	Requires two parameters: <i>eps</i> and <i>min_samples</i>	Requires one parameter: number of clusters (K)

Evaluation Metrics

Category	Metric	Description / Purpose	Formula / Key Concept
Classification Metrics	Accuracy	Measures how many predictions were correct out of total predictions. Misleading with imbalanced data.	Accuracy = Correct Predictions / Total Predictions
	Precision	Fraction of correctly predicted positive instances. Useful when False Positives are costly.	Precision = TP / (TP + FP)
	Recall (Sensitivity)	Fraction of actual positives correctly identified. Important when False Negatives are costly.	Recall = TP / (TP + FN)
	F1 Score	Harmonic mean of Precision and Recall. Balances both metrics.	$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
	Logarithmic Loss (Log Loss)	Measures model uncertainty; penalizes incorrect probabilities. Lower = better.	$\text{Log Loss} = -(1/N) \sum \sum y_{ij} \log(p_{ij})$
	ROC-AUC Score	Evaluates classification thresholds; higher AUC = better model discrimination.	AUC = Area under ROC curve (TPR vs FPR)
	Confusion Matrix	Summarizes predictions into TP, TN, FP, FN. Useful for visualizing errors.	Matrix with Actual vs Predicted values

Evaluation Metrics

Regression Metrics	Description / Purpose	Formula / Key Concept
Mean Absolute Error (MAE)	Average of absolute differences between predicted and actual values.	$MAE = (1/N) \sum$
Mean Squared Error (MSE)	Average of squared differences; penalizes large errors.	$MSE = (1/N) \sum (y_i - \hat{y}_i)^2$
Root Mean Squared Error (RMSE)	Square root of MSE; interpretable in same units as target variable.	$RMSE = \sqrt{(\sum (y_i - \hat{y}_i)^2 / N)}$
Root Mean Squared Logarithmic Error (RMSLE)	Penalizes underestimation; useful for wide-range data.	$RMSLE = \sqrt{(\sum (\log(y_i+1) - \log(\hat{y}_i+1))^2 / N)}$
R ² (Coefficient of Determination)	Proportion of variance explained by the model.	$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$

Evaluation Metrics

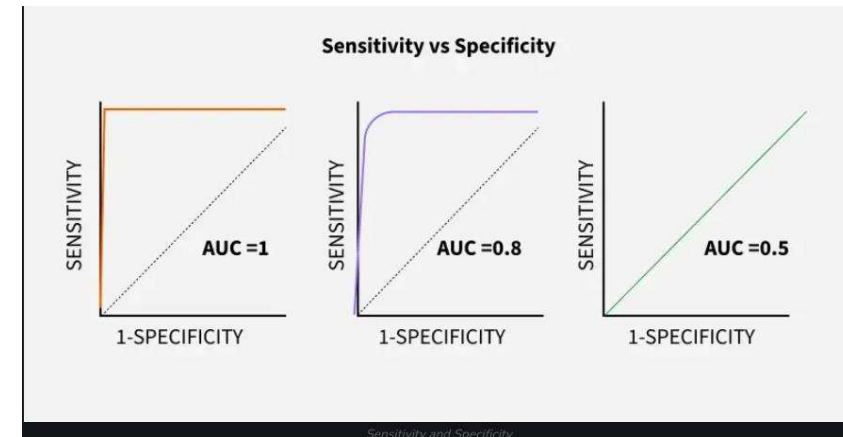
Regression Metrics	Description / Purpose	Formula / Key Concept
Mean Absolute Error (MAE)	Average of absolute differences between predicted and actual values.	$MAE = (1/N) \sum$
Mean Squared Error (MSE)	Average of squared differences; penalizes large errors.	$MSE = (1/N) \sum (y_i - \hat{y}_i)^2$
Root Mean Squared Error (RMSE)	Square root of MSE; interpretable in same units as target variable.	$RMSE = \sqrt{(\sum (y_i - \hat{y}_i)^2 / N)}$
Root Mean Squared Logarithmic Error (RMSLE)	Penalizes underestimation; useful for wide-range data.	$RMSLE = \sqrt{(\sum (\log(y_i+1) - \log(\hat{y}_i+1))^2 / N)}$
R^2 (Coefficient of Determination)	Proportion of variance explained by the model.	$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$

Evaluation metrics

Clustering Metrics	Description / Purpose	Formula / Key Concept
Silhouette Score	Measures how well samples fit their cluster compared to others. Higher = better.	$(b - a) / \max(a, b)$
Davies–Bouldin Index	Measures average similarity between clusters; lower = better separation.	$DBI = (1/N) \sum \max_{i \neq j} ((\sigma_i + \sigma_j) / d(c_i, c_j))$

Sensitivity vs Specificity

- **Sensitivity (Recall / True Positive Rate):**
 - Measures the proportion of *actual positives* correctly identified.
 - Focuses on minimizing **False Negatives (FN)**.
 - Formula:
Sensitivity = $TP / (TP + FN)$
 - Example:
If a fraud detection model correctly detects 90 out of 100 frauds →
Sensitivity = $90 / (90 + 10) = 0.90$
 - **High Sensitivity → Few missed positive cases**
- **Specificity (True Negative Rate):**
 - Measures the proportion of *actual negatives* correctly identified.
 - Focuses on minimizing **False Positives (FP)**.
 - Formula:
Specificity = $TN / (TN + FP)$
 - Example:
If a model correctly labels 950 of 1000 non-frauds →
Specificity = $950 / (950 + 50) = 0.95$
 - **High Specificity → Few false alarms**
- **On ROC Curve:**
 - AUC (Area Under Curve) shows the balance between Sensitivity & Specificity.
 - **AUC = 1.0** → Perfect Model
 - **AUC = 0.8** → Good Balance
 - **AUC = 0.5** → Random Guess



Aspect	Sensitivity	Specificity
Definition	Detects actual positives correctly	Detects actual negatives correctly
Focus	True Positives	True Negatives
Error Controlled	False Negatives	False Positives
Useful When	Missing positives is risky (e.g., medical, fraud detection)	False alarms are costly (e.g., credit scoring, spam filters)
Increase When	FN decreases	FP decreases

ROC –AUC Curve

What is ROC Curve?

ROC(Receiver Operating Characteristic) curve is used to evaluate a binary classification model's performance.

It plots:

- True Positive Rate (TPR / Sensitivity) → Correctly predicted positives.
- False Positive Rate (FPR) → Incorrectly predicted negatives as positives.
- Specificity = $1 - \text{FPR}$

Goal: Show the trade-off between *Sensitivity* and *Specificity* at various threshold values.

What is AUC?

AUC (Area Under the Curve) measures the overall performance of the classifier.

Higher AUC → Better model at distinguishing positive and negative cases.

AUC Values Interpretation:

- 1.0 → Perfect Model
- 0.8 → Good Model
- 0.5 → Random Guess

How it Works:

Choose one positive and one negative data point.

Check if the positive has a higher predicted probability.

Repeat for all pairs → AUC measures how well the model ranks positives higher than negatives.

When to Use:

Balanced datasets where both positive and negative classes are important.

When false positives and false negatives have similar cost.

For imbalanced datasets, use **Precision–Recall Curve** instead.

Model Performance:

- **High AUC (~1):** Model clearly

BIAS VARIANCE TRADEOFF

Concept

Prediction errors arise from **bias** and **variance**.

A **trade-off** exists between minimizing both — finding the **best regularization constant** balances them.

Goal: **Avoid underfitting** (high bias) and **overfitting** (high variance).

Bias

Difference between model predictions and true values.

High Bias → Underfitting (oversimplified model, e.g., linear fit).

Variance

Measures model sensitivity to training data.

High Variance → Overfitting (too complex model, poor generalization).

Trade-Off

High Bias → Low Variance (simple model)

Low Bias → High Variance (complex model)

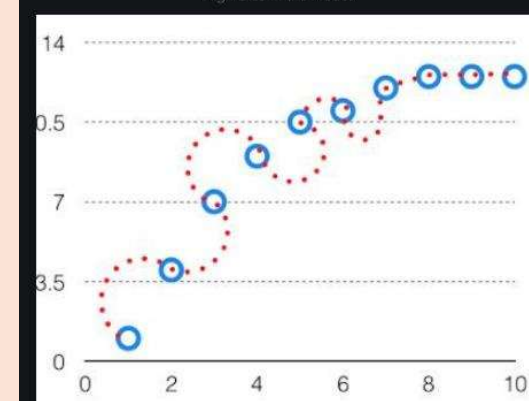
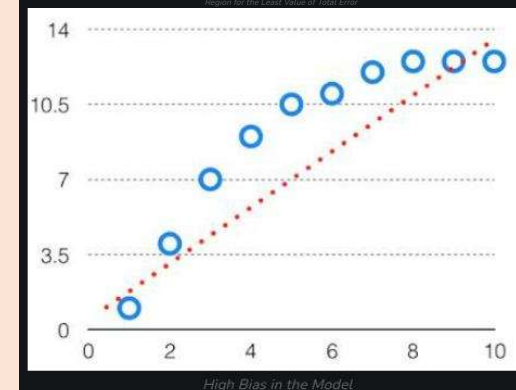
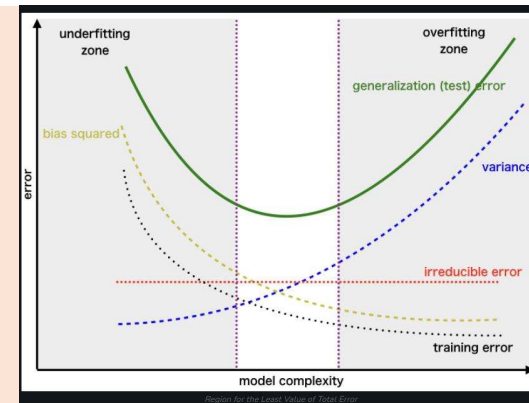
Optimal point minimizes total error:

Total Error = Bias² + Variance + Irreducible Error

The **best model** lies where training and test errors are both low.

Key Idea:

Find the sweet spot — not too simple, not too complex — for the least total error.



MODEL SELECTION

Why Model Selection Matters

Determines **accuracy, efficiency, and reliability** of predictions.

Poor model → **overfitting** (too complex) or **underfitting** (too simple).

Goal: Learn patterns **effectively** without being too simple or too complex.

Influences **training time**, dataset handling, and computational costs.

Steps in Model Selection

Understand the Problem & Data

Regression → Predict continuous values (e.g., house prices)

Classification → Predict categories (e.g., spam vs non-spam)

Clustering → Group similar data points (e.g., customer segmentation)

Examine dataset: missing values, categorical/numerical features, distribution

Select Suitable Models

Regression: Linear Regression, Decision Trees, Random Forest, Neural Networks

Classification: Logistic Regression, SVM, k-NN, Neural Networks

Clustering: k-Means, Hierarchical, DBSCAN

Model Evaluation

Split data: Training set (learn patterns) & Testing set (evaluate)

Use **k-fold cross-validation** for unbiased evaluation

Metrics:

Regression → MSE, MAE, R^2

Classification → Accuracy, Precision, Recall, F1-score

Grid Search

Systematically tries all hyperparameter combinations

Pros: Finds optimal parameters

Cons: Computationally expensive

Random Search

Randomly selects a subset of parameter combinations

Faster than grid search with comparable results

Bayesian Optimization

Uses probability models to **predict best hyperparameters**

Efficient and often better than grid/random search

Cross-Validation Based Selection

Evaluates models on multiple data splits

Averages results → reduces overfitting risk

Ensures good performance on unseen data

Key Takeaways

Proper model selection **improves accuracy, efficiency, and reliability**

Combine **performance metrics, dataset characteristics, and computational resources**

Use **systematic hyperparameter tuning** to find the best model

PRACTICE QUESTIONS Unit III

- Write difference between Classification and regression. (ii) Write five real life application of classification.
- Explain Logistic Regression with example. (ii) Explain Support Vector machine with example.
- Explain Decision Tree with example. (ii) Explain Random Forest with example.
- Explain terms – Classifier, Classification Model, Labels, Features, KNN.
- Explain the complete pipeline for building a machine learning model from scratch. Include data preprocessing, model selection, training, evaluation, and deployment.
- Discuss the importance of splitting data into training, validation, and testing sets. How does each contribute to model development and evaluation?
- Describe various techniques to improve model accuracy and generalization. Include examples like regularization, cross-validation, and feature engineering.
- Explain the working of a Random Forest Classifier. How does it overcome the limitations of a single decision tree?
- Discuss the role of feature importance in Random Forest. How can it be used for feature selection and model interpretation?

NUMERICAL QUESTION Unit III

Datasets have total 20 cats and dog's images. Given that machine learning model trained on Binary classification gives following results. [1] Predicted cat and actual cat are same =10, [2] Predicted not cat and actual is dog = 6 [3] Predicted cat and actual is dog = 3. [4] Predicted not cat and actual is cat =1.

(i) Draw Confusion matrix (ii) Define terms and find value of True Positives, False Positives, True negative, False Negatives (iii) Define terms and find value of Accuracy, Precision, Recall and F1-Score for model.

Datasets have total 35 cats and dog's images. Given that machine learning model trained on Binary classification gives following results. [1] Predicted cat and actual cat are same =10, [2] Predicted not cat and actual is dog = 11 [3] Predicted cat and actual is dog = 5. [4] Predicted not cat and actual is cat =9.

(i) Draw Confusion matrix (ii) Define terms and find value of True Positives, False Positives, True negative, False Negatives (iii) Define terms and find value of Accuracy, Precision, Recall and F1-Score for model.

CASE STUDIES Unit III

Case Study:

A healthcare startup is building a model to predict whether a patient has diabetes (Yes/No). Later, they expand the system to classify patients into one of four disease categories. Explain the difference between binary and multiclass classification and how model design changes accordingly.

- **Binary classification** involves two classes (e.g., diabetic vs non-diabetic). Algorithms like logistic regression or SVM are commonly used.
- **Multiclass classification** involves more than two classes (e.g., diabetes, hypertension, asthma, none).
- Binary models use a single decision boundary, while multiclass models often use one-vs-rest or softmax-based approaches.
- Evaluation metrics shift from binary (accuracy, precision, recall) to macro/micro-averaged scores in multiclass settings.
- Model complexity increases with multiclass tasks, requiring more data and careful handling of class imbalance.

Case Study:

A bank uses decision trees to classify loan applications as approved or rejected, and to predict loan amounts. Explain how decision trees handle both classification and regression tasks.

- For **classification**, decision trees split data based on criteria like Gini impurity or entropy to separate classes.
- For **regression**, they minimize variance within splits to predict continuous values.
- Trees are built top-down, selecting the best feature at each node.
- They are easy to interpret and visualize, but prone to overfitting.
- Pruning techniques and depth control help improve generalization.

Practice questions Unit IV

Clustering in Machine Learning

Explain the concept of clustering in machine learning. How does it differ from classification, and what are its key applications?

Describe the K-Means clustering algorithm in detail. Include its working steps, objective function, limitations, and methods to choose the optimal number of clusters.

Discuss the DBSCAN algorithm. How does it identify clusters and noise? Compare its strengths and weaknesses with K-Means.

Explain hierarchical clustering. Differentiate between agglomerative and divisive approaches. How is a dendrogram used in this method?

Evaluate clustering models using internal and external metrics. Discuss the role of silhouette score, Davies–Bouldin index, and adjusted Rand index.

Dimensionality Reduction

What is dimensionality reduction? Why is it important in machine learning? Discuss the challenges of working with high-dimensional data.

Explain the Principal Component Analysis (PCA) technique. Include its mathematical foundation, steps, and how it helps in reducing dimensionality.

Describe Linear Discriminant Analysis (LDA). How does it differ from PCA in terms of objective and application?

Compare and contrast feature selection and feature extraction. Provide examples of techniques used for each and discuss their impact on model performance.

Discuss the role of eigenvalues and eigenvectors in PCA. How do they contribute to identifying principal components?

Numerical Questions Unit IV

K-Means Clustering – Solved Example

- Suppose that the data mining task is to cluster points into three clusters,
- where the points are
- $A1(2, 10)$, $A2(2, 5)$, $A3(8, 4)$, $B1(5, 8)$, $B2(7, 5)$, $B3(6, 4)$, $C1(1, 2)$, $C2(4, 9)$.
- The distance function is Euclidean distance.
- Suppose initially we assign $A1$, $B1$, and $C1$ as the center of each cluster, respectively.

K Means Clustering Solved Example K Means Clustering Algorithm in Machine Learning by Vishesh Pruthi

K-Means Clustering Algorithm – Solved Example

- Use K Means clustering to cluster the following data into two groups.
- Data Points: $\{ 2, 4, 10, 12, 3, 20, 30, 11, 25 \}$
- The distance function used is Euclidean distance.
- Initial cluster centroid are $M1 = 4$ and $M2 = 11$.

Numerical Questions Unit IV

DBSCAN Clustering Algorithm Solved Example – 1

- Apply the DBSCAN algorithm to the given data points and
 - Create the clusters with
 - $\text{minPts} = 4$ and
 - $\text{epsilon} (\epsilon) = \underline{1.9}$.
- Data Points:
- | | |
|-------------|-------------|
| P1: (3, 7) | P2: (4, 6) |
| P3: (5, 5) | P4: (6, 4) |
| P5: (7, 3) | P6: (6, 2) |
| P7: (7, 2) | P8: (8, 4) |
| P9: (3, 3) | P10: (2, 6) |
| P11: (3, 5) | P12: (2, 4) |

DBSCAN Clustering Algorithm Solved Example – 2

Apply the DBSCAN algorithm with similarity threshold of 0.8 (using the similarity matrix) to the given data points and $\text{MinPts} \geq 2$. (Minimum required points in a cluster) what are core, border and noise (outliers) in the set of points given in table.

	P1	P2	P3	P4	P5
P1	1.00	0.10	0.41	0.55	0.35
P2	0.10	1.00	0.64	0.47	0.98
P3	0.41	0.64	1.00	0.44	0.85
P4	0.55	0.47	0.44	1.00	0.76
P5	0.35	0.98	0.85	0.76	1.00

Numerical Questions Unit IV

Principle Component Analysis – Solved Example

- Given the data in Table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Linear Discriminant Analysis - Solved Example

- Compute the Linear Discriminant projection for the following two dimensional dataset.

Samples for class ω_1 : $\mathbf{X}_1 = (x_1, x_2) = \{(4,2), (2,4), (2,3), (3,6), (4,4)\}$

Sample for class ω_2 : $\mathbf{X}_2 = (x_1, x_2) = \{(9,10), (6,8), (9,5), (8,7), (10,8)\}$

Case Studies Unit IV

- **A hospital wants to group patients based on symptoms to improve diagnosis. How can clustering help in this scenario?**
 - Identifies hidden patterns in patient data
 - Groups similar symptom profiles
 - Aids in early disease detection
 - Supports personalized treatment plans
 - Reduces diagnostic errors
 - Helps allocate medical resources efficiently
 - Enables targeted health campaigns
 - Improves patient monitoring
 - Facilitates research on symptom clusters
- **A travel company wants to categorize destinations based on traveler preferences. Which clustering method is suitable and why?**
 - K-Means for clear, predefined categories
 - DBSCAN for irregular, density-based preferences
 - Handles both popular and niche destinations
 - Reveals hidden travel trends
 - Improves recommendation systems
 - Supports personalized travel packages
 - Enhances user experience
 - Helps in market segmentation
 - Reduces search complexity

Practice questions Unit V

Model Building & Training

Explain the complete process of building a machine learning model from scratch. Include data preprocessing, model selection, training, evaluation, and deployment.

Discuss the importance of splitting data into training and testing sets. How does this impact model performance and generalization?

Describe various techniques used to improve model accuracy and generalization. Include examples such as cross-validation, regularization, and feature engineering.

Random Forest Classifier

Explain the working of a Random Forest Classifier. How does it overcome the limitations of a single decision tree?

Discuss the role of feature importance in Random Forest. How can it be used for feature selection and model interpretation?

Clustering Algorithms

Describe the K-Means clustering algorithm. Include its steps, objective function, limitations, and methods to choose the optimal number of clusters.

Explain the DBSCAN algorithm. How does it identify clusters and noise? Compare its performance with K-Means in handling non-spherical clusters.

Write the steps to implement K-Means and DBSCAN from scratch. Highlight the mathematical intuition behind each algorithm.

Performance Metrics

Define and compare accuracy, precision, recall, sensitivity, and specificity. In which scenarios are each of these metrics most useful?

Explain the concept of confusion matrix. How can it be used to derive various performance metrics for classification models?



Case Studies Unit V

- **Case:** A healthcare startup wants to build a model to predict patient readmission. What steps should they follow to build a machine learning model?
 - Collect historical patient data
 - Clean and preprocess data (handle missing values, encode categories)
 - Perform exploratory data analysis
 - Select relevant features
 - Choose a suitable algorithm (e.g., logistic regression, Random Forest)
 - Split data into training and testing sets
 - Train the model on training data
 - Evaluate using performance metrics
 - Tune hyperparameters for optimization
 - Deploy the model in a hospital system
- **Case:** A bank wants to predict loan defaults. How should they approach model training and testing?
 - Gather customer financial and credit history
 - Preprocess and normalize data
 - Split data into training and testing sets
 - Train model using supervised learning
 - Use cross-validation to avoid overfitting
 - Evaluate using accuracy, precision, recall
 - Adjust model parameters
 - Test on unseen data
 - Compare multiple models
 - Finalize and deploy the best-performing model